



合肥工业大学
计算机与信息学院

计算机科学与技术专业新生研讨课
课程报告

姓 名： 姚雅然

专业班级： 计算机科学与技术 23-3 班

学 号： 2023212228

2023 年 1 月 2 日

目录

前言	3
1. QT 的简单介绍	4
1.1 什么是 QT?	4
1.2 QT 的优点	5
1.3 QT 的组成	6
2. QT 的安装	8
3. QT 界面设计使用	13
3.1 创建一个 QT 项目	13
3.2 案例分析	16
3.3 设计 QT 界面	21
3.4 信号与槽	24
3.5 案例总结	28
4. QDebug 的使用	36
后记	37

前言

说来惭愧，作为刚刚接触计算机学科的小萌新，对于 QT 这门技术，我一开始学习的原因是为了完成对期末的课设作业的可视化，机缘巧合之下了解到运用 QT 设计交互界面也许更为快捷方便。而且 QT 是基于 C++ 语言进行开发的，想来临时抱抱佛脚，学习一些入门的操作应该不会非常困难，这也足以让我应付期末课设作业的可视化实现了。

但真正查找资料，跟着网上的教程一点点学起，又向有经验技术的同学请教，我才越发认识到 QT 技术的深度与广度。从一开始抱着浅浅接触学习去应付作业的心态，到现在对这门技术有着较为浓厚的兴趣，不仅是因为我自身很喜欢美工设计，还因为我在学习的过程中对 C++ 语言运用有更深的心得体会，感受到计算机学科的魅力所在，对于人机交互也有了进一步的理解。

所以这一份资料，是我作为一个新手初步学习了解 QT 技术的一些较为浅显的学习成果和心得体会。

1. QT 的简单介绍

1.1 什么是 QT ？

QT 是一个跨平台的 C++ 图形用户界面库，目前包括 Qt Creator，Qt Embedded，Qt Designer 支持所有 Linux / Unix 系统，还支持 Windows 平台。【1】

QT 是面向对象的 GUI 程序设计。操作系统提供了创建界面元素所需要的函数，依次调用不同功能的函数就可以创建出界面元素，但操作系统提供的原生函数无法直接映射到界面元素。根据面向对象程序设计思想，将界面元素抽象为类，GUI 界面的创建过程就是组合不同界面元素对象的过程。【2】

那么 QT 的本质就是利用面向对象程序设计思想开发的一套 GUI 组件库，QT 将不同操作系统的 GUI ①细节封装于类内部，并提供了一套跨平台的类用于 GUI 程序开发，它遵循经典的 GUI 程序开发模式②。

①注：GUI 中文名称为图形用户界面，是指采用图形方式显示的计算机操作用户界面，是计算机与其使用者之间的对话接口，是计算机系统的重要组成部分。【3】

②注：GUI 程序开发原理：首先，GUI 程序在运行时会创建一个消息队列，然后系统内核将用户操作翻译为对应的程序消息，而程序在运行过程中需要实时处理消息队列中的消息。当队列中没有消息时，程序将处于停滞状态。

GUI 程序开发过程：1. 创建窗口、窗口元素 GUI 界面。2. 在消息处理函数中根据程序消息做出不同相应。【4】

1.2 QT 优点

Qt 是一个跨平台的 C++ 图形用户界面应用程序框架，提供给应用程序开发者建立艺术级的图形用户界面所需的所用功能。Qt 很容易扩展，并且允许真正地组件编程。Qt 与 GTK、KDE、MFC，OWL，VCL，ATL 是一样的图形界面库。

1) 优良的跨平台特性

Qt 支持下列操作系统：Microsoft Windows，Linux，Solaris，SunOS，HP - UX，Digital UNIX (OSF /1, Tru64)，Irix，FreeBSD，BSD / OS，SCO，AIX，OS390，QNX

2) 面向对象

Qt 的良好封装机制使得 Qt 的模块化程度非常高，可重用性较好，对于用户开发来说非常方便。Qt 提供了一种称为 signals / slots 的安全类型来替代 callback，使得各个元件之间的协同工作变得十分简单。

3) 丰富的 API

Qt 包括多达 250 个以上的 C++ 类，还提供基于模板的 collections，serialization，file，I/O device，directory management，date / time

4) 支持 2D/3D 图形渲染，支持 OpenGL

5) 大量的开发文档

6) XML 支持【5】

1.3 QT 的组成

QT 开发工具包含 Qt Creator、Qt Designer、Qt Linguist、Qt Assistant、Qmake。

1) Qt Creator

Qt Creator 是用于 Qt 开发的轻量级跨平台集成开发环境。

2) Qt Designer

Qt Designer 是强大的拖曳式图形化用户界面排版和设计工具，它的功能如下：

- A. 支持表单和对话框的创建，可即时预览
- B. 与 QT 版面系统集成
- C. 宏大的标准 widgets 集
- D. 支持客户定制的 widgets 和对话框
- E. 与 Microsoft Visual Studio . NET 无缝集成

Qt Designer 的优势也很明显，它大大加快了界面的设计过程，支持所有平台上的本地外观感觉，这使得开发者能在自行选择的工作环境中充分发挥其现有技能

3) Qt Linguist

Qt Linguist 一整套工具，支持对 Qt 应用作快捷无误的翻译，是一组能理顺国际化工作流的工具，它的功能如下：

- A. 采集所有的用户界面文本并以一个简洁的窗口将其展现给人工译者

B. 支持所有语言

C. 从单一应用的二进制程序内部提供同时多语言支持及同时多写入系统

Qt Linguist 大大加快了翻译/本地化进程，它与 Qt 的语言敏感排版引擎协同，以创建与语言不相关的简洁一致的界面，能够轻松应对国际市场

4) Qt Assistant

Qt Assistant 是可定制可重发布的帮助文件和文档阅读器，其功能如下：

A. 简单明快的 web 浏览器般导航、书签和文档文件连接

B. 支持富文本 HTML

C. 全文本关键词查阅

D. 可定制并随 Qt 供应

运用 Qt Assistant 则无需再从头开始构建帮助系统，能充分利用现有的 HTML 技能，以方便搜寻和导航的格式向最终用户提供文档

5) Qmake

Qmake 是跨平台的 makefile 生成器，它可以

A. 读取工程源码，生成依赖关系树，生成平台相关工程和 makefiles

B. 与 Visual Studio 及 Xcode 集成

所以使用 Qmake 无需担忧跨平台编译，降低对 makefile 手工构建的需求度【6】

2. QT 的安装

*注：由于 QT6 的版本安装不能离线安装，需要登录注册，以下演示的是 QT5.14.2 的安装

Qt 官网有一个专门的资源下载网站，所有的开发环境和相关工具都可以从这里下载，具体地址是：[Qt Downloads](#)

Qt Downloads			
Qt Home Bug Tracker Code Review Planet Qt Get Qt Extensions			
Name	Last modified	Size	Metadata
■ snapshots/	02-Dec-2021 13:12	-	
■ online/	19-Nov-2020 14:24	-	
■ official_releases/	04-Feb-2021 08:00	-	
■ new_archive/	29-Jan-2021 15:13	-	
■ ministro/	20-Feb-2017 10:32	-	
■ linguist_releases/	26-Mar-2019 07:49	-	
■ learning/	24-Feb-2021 15:09	-	
■ development_releases/	06-May-2021 07:48	-	
■ community_releases/	23-Feb-2017 07:29	-	
■ archive/	10-Dec-2020 12:52	-	
■ timestamp.txt	05-May-2022 16:00	11	Details

official_releases 和 archive 文件夹都有最新的 Qt 开发环境安装包，看了很多教程，一般都是在 archive 文件夹里下载。点击进入 archive 文件夹：

Qt Downloads			
Qt Home Bug Tracker Code Review Planet Qt Get Qt Extensions			
Name	Last modified	Size	Metadata
↑ Parent Directory		-	
■ vsaddin/	05-Jan-2022 13:23	-	
■ qtcreator/	03-Dec-2021 10:18	-	
■ qt/	04-Apr-2022 12:09	-	
■ online_installers/	24-Feb-2022 12:54	-	
■ additional_libraries/	03-Mar-2021 10:18	-	

For Qt Downloads, please visit qt.io/download

Qt® and the Qt logo is a registered trade mark of The Qt Company Ltd and is used pursuant to a license from The Qt Company Ltd.
All other trademarks are property of their respective owners.
The Qt Company Ltd, Bertel Jungin aukio D3A, 02600 Espoo, Finland. Org. Nr. 2637805-2
[List of official Qt-project mirrors](#)

点进qt文件夹，包含了所有的 Qt 版本，从 1.0 到目前的 6.3。

Qt Downloads

Qt Home Bug Tracker Code Review Planet Qt Get Qt Extensions

Name	Last modified	Size	Metadata
↑ Parent Directory		-	
6.3/	08-Apr-2022 12:09	-	
6.2/	05-Apr-2022 13:38	-	
6.1/	01-Sep-2021 10:42	-	
6.0/	04-May-2021 07:38	-	
5.15/	05-Apr-2022 13:39	-	
5.14/	31-Mar-2020 09:14	-	
5.13/	31-Oct-2019 07:17	-	
5.12/	25-Nov-2021 08:02	-	
5.9/	16-Dec-2019 15:00	-	
5.1/	28-Jan-2021 16:26	-	
5.0/	28-Jan-2021 16:48	-	
4.8/	09-Mar-2021 10:54	-	
4.7/	28-Jan-2021 16:25	-	
4.6/	28-Jan-2021 17:11	-	
4.5/	29-Jan-2021 15:07	-	
4.4/	28-Jan-2021 16:01	-	
4.3/	28-Jan-2021 16:06	-	

进入 5.14 文件夹，可以看到各种子版本：

Qt Downloads

Qt Home Bug Tracker Code Review Planet Qt Get Qt Extensions

Name	Last modified	Size	Metadata
↑ Parent Directory		-	
5.14.2/	31-Mar-2020 10:28	-	
5.14.1/	27-Jan-2020 07:27	-	
5.14.0/	11-Dec-2019 15:49	-	

点进5.14.2/：

Qt Downloads

Qt Home Bug Tracker Code Review Planet Qt Get Qt Extensions

Name	Last modified	Size	Metadata
↑ Parent Directory		-	
submodules/	31-Mar-2020 09:27	-	
single/	31-Mar-2020 10:10	-	
qt-opensource-windows-x86-5.14.2.exe	31-Mar-2020 10:16	2.3G	Details
qt-opensource-mac-x64-5.14.2.dmg	31-Mar-2020 10:16	2.6G	Details
qt-opensource-linux-x64-5.14.2.run	31-Mar-2020 10:14	1.2G	Details
md5sums.txt	31-Mar-2020 10:32	207	Details

各个子模块源代码
QT完整模块源代码
QT基于Windows版本
QT基于MAC版本
QT基于Linux版本

点击QT基于Windows版本下载。

qt-opensource-windows-x86-5.14.2.exe命名规则：

opensource 是指开源版本；

windows 是指开发环境的操作系统；

x86 是指 32 位系统；

5.14.2 是 Qt 版本号。

←

←

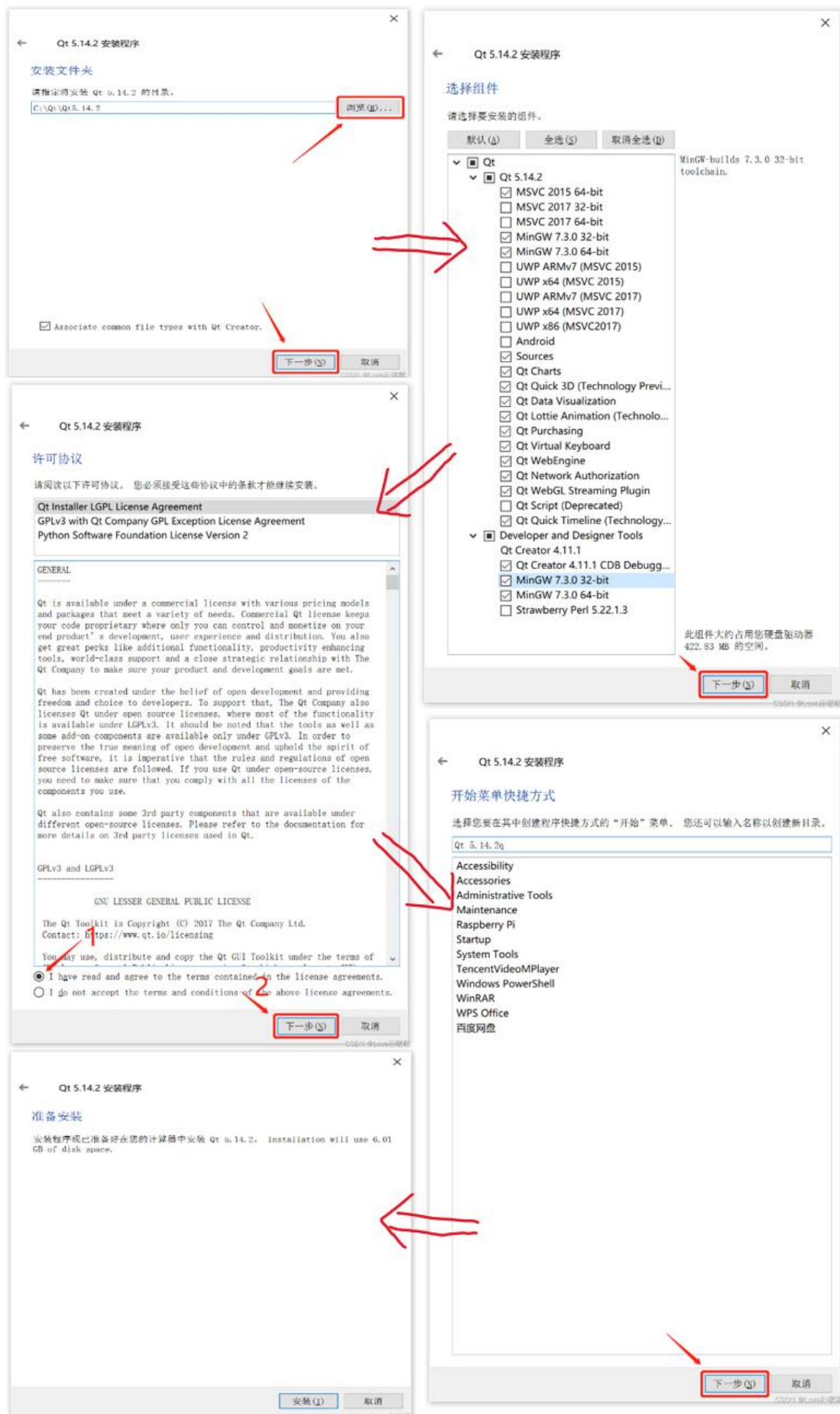
*注：在点击安装前可先断开网络，这样可以离线下载，避免被要求注册或登录 QT 账号

1、管理员权限运行qt-opensource-windows-x86-5.14.2.exe

 qt-opensource-windows-x86-5.14.2.exe

2、点击next





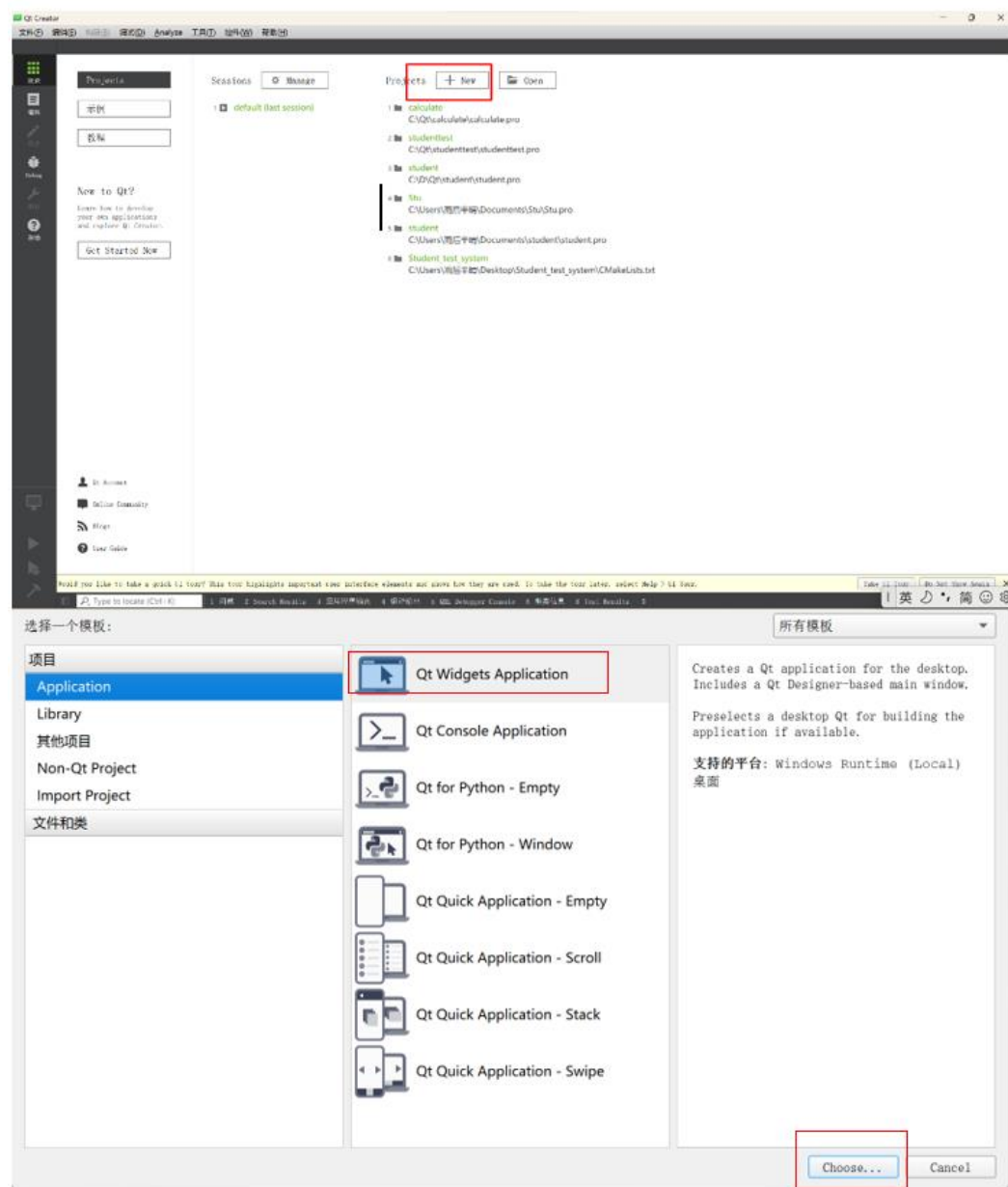


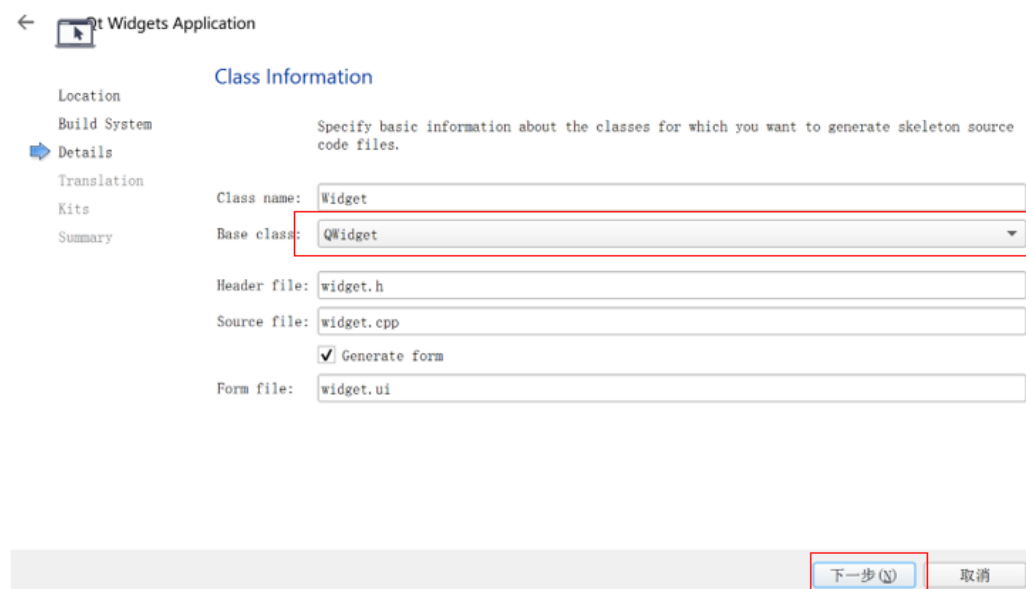
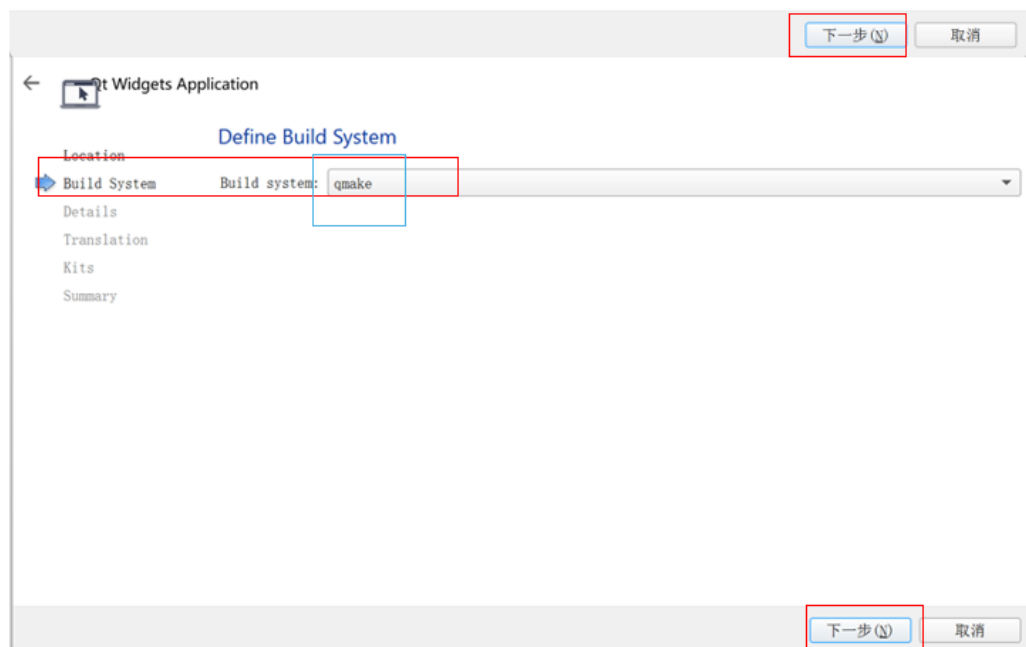
【7】

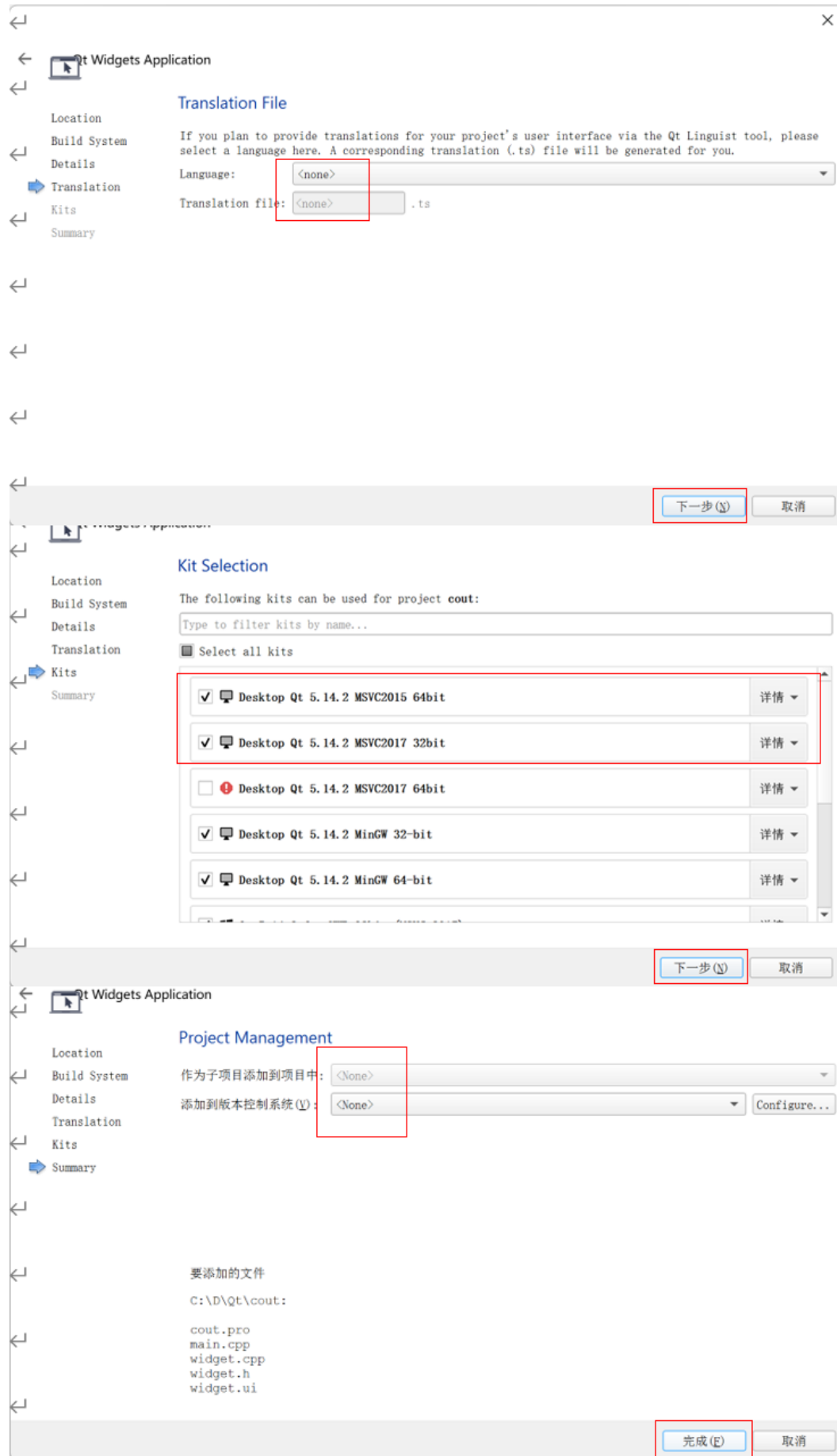
3. QT 界面设计使用

接下来我会以设计一个四则计算器为例，对 QT 界面设计的功能运用做一些示范。

3.1 创建一个 QT 项目



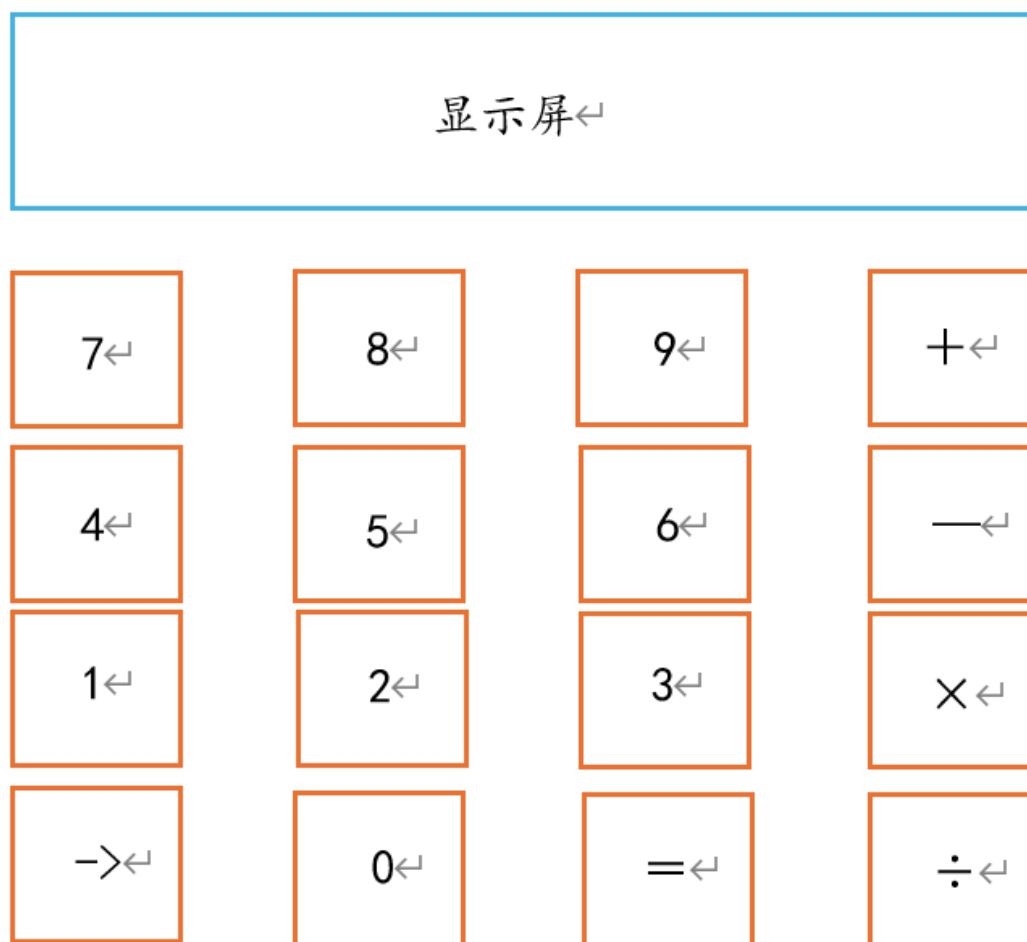




3.2 案例分析

像我们手机上的计算器一样，有数字 0~9，这里我们进行简单的四则整数运算，则有“+”“-”“×”“÷”“=”，还有一个清零的操作“->”，这些都是按键，而我们还需要一个显示屏来显示计算的过程，这里的显示屏就是我们输出的文本窗口。

那确定好我们界面所需要的元素，我们可以对界面的排版进行一个简单的设计。



比如，当我们计算 $5 \times 6 = 30$ 时，应该是：点击按键“5”->显示屏出现“5”->点击按键“×”->显示屏变为“5×”->点击按键“6”->显示屏变为“5×6”->点击按键“=”->显示屏变为“30”。

做好界面设计后，我们需要思考一下控制运行的代码。由于我们对于按键的点击是一下一下的，那么，当我们输入两位数或三位数，甚至多位数时时，我们无法一下子得到完整的一个数字，而是这个数字每个数位上数的拆分（比如，输入“30”时，我们实际得到的是“3”和“0”），所以我这里设置了一个 `arr[100]` 的数组，用于存放按键结果。我们可以给每个按键一个数字作为编号，当按键被点击是，相对应的编号存入 `arr` 数组中。为了方便记忆，按键“0”~“9”的编号分别是 0~9，而按键“=”“+”“-”“×”“÷”编号分别为-1，-2，-3，-4，-5①。



```
1  #ifndef WIDGET_H
2  #define WIDGET_H
3
4  #include <QWidget>
5  #include<iostream>
6  #include<cstring>
7  #include<cmath>
8  #include<QDebug>
9  using namespace std;
10
11  QT_BEGIN_NAMESPACE
12  namespace Ui { class Widget; }
13  QT_END_NAMESPACE
14
15  class Widget : public QWidget
16  {
17      Q_OBJECT
18
19  public:
20      int m=0; //数据在arr[100]中存放的位置
21      int arr[100];
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53  void Widget::on_num4_clicked()
54  {
55      arr[m]=4;
56      m++;
57      str+="4";
58      QString str_ = QString::fromStdString(str);
59      ui->result->setText(str_);
60  }
61  void Widget::on_num5_clicked()
62  {
63      arr[m]=5;
64      m++;
65      str+="5";
66      QString str_ = QString::fromStdString(str);
67      ui->result->setText(str_);
68  }
```

设置好按键编号，对于运算符目前并不需要进行加工，而输入的数字则需要一一重新整合起来（比如，在 `arr` 数组中，在两个代表运算符编号中间有编号“3”和编号“0”挨在一起，那么操作者其实一开始想要输入的是数字“30”，则我们需要把数字“3”和数字“0”整合在一起变成“30”），代码如下：

```
133 void Widget::on_func_clicked()
134 {
135     arr[m] = -1;
136     m++;
137     int b = 0;
138     int a[100]; //存放处理过后的数据的数组
139     int x = 0;
140     for(int i = 0; i < m; i++){
141
142         if(arr[i] >= -5 && arr[i] < 0){ //遇到运算符
143             int y = 0;
144             int num = 0; //处理后的数据
145
146             for(int j = i - 1; j >= i - b; j--){ //将零散的数字组成一个新的多位数
147                 num = num + arr[j] * pow(10, y);
148                 y++;
149             }
150             a[x] = num; //将处理过的数据放入数组a中
151             x++;
152             a[x] = arr[i]; //arr[i]存放的数据代表某一运算符，直接赋值到数组a中，无需处理
153             x++;
154             b = 0;
155         }
156         else if(arr[i] >= 0){ //计算上一运算符到下一运算符之间数字个数
157             b++;
158         }
159     }
}
```

现在数组 `a` 中存放的就是处理过后的数据。如果我们进行运算的是“ 5×6 ”，那么数组 `a` 中存放的应该是“5, -4, 6, -1”，我们按数组顺序，直接读取数字和运算符编号并没有什么问题。但是，如果我们计算的是“ $5 + 5 \times 6$ ”呢？此时，数组 `a` 中存放的应该是“5, -2, 5, -4, 6, -1”，若我们还是按数组顺序读取数字和运算符编号进行运算，这条式子的运算顺序便从先乘后加变成先加后乘了，运算结果自然也就从 25 变成 60 了。

所以，鉴于“ $\times$ ”“ $\div$ ”优先于“ $+$ ”“ $-$ ”，我们需要对数据再一次处理。方法就是在数组 `a` 中搜索代表“ $\times$ ”“ $\div$ ”的编号，先将编号前后的数字进行运算并重新存放进数组 `a` 中（比如，遍历到数

组 a 中的一段数据为“5, -4, 6”，那么计算 $5 \times 6 = 30$ ，将 30 赋值到“-4”的位置，然后将“5”和“6”用“-10”取代，则这段数据变为“-10, 30, -10”。

```
160 for(int i=1;i<x-1;i++){
161     if(a[i]==-4){ //乘法运算
162         a[i]=a[i-1]*a[i+1]; //“x”号前后数字相乘计算
163         a[i-1]=-10;
164         a[i+1]=-10;
165     }
166     else if(a[i]==-5){ //除法运算
167         a[i]=a[i-1]/a[i+1]; //“÷”号前后数字相乘计算
168         a[i-1]=-10;
169         a[i+1]=-10;
170     }
171 }
```

再次处理后，数组 a 中的数据从“5, -2, 5, -4, 6, -1”变成了“5, -2, -10, 30, -10, -1”。但是对于数组 a 的处理并没有结束，因为在运算“ 5×6 ”后，我们要接着计算“ $5 + 30$ ”，“ $5 + 30$ ”若存放在数组 a 中，应该是“5, -2, 30, -1”，但现在，对比之下，数组 a 中存放的数据在“-2”与“30”中间出现了“-10”的干扰，所以我们要对数组进行第三次处理，即将所有的“-10”往后移动，即数据排序变成“5, -2, 30, -1, -10, -10”。

```
172 for(int i=0;i<x-1;i++){
173     for(int j=0;j<x-i-1;j++){
174         if(a[j]==-10){
175             swap(a[j],a[j+1]);
176         }
177     }
178 }
```

这样，我们按数组顺序，读取数字和运算符编号进行运算，到“-1”时停止读取，就可以得到计算结果了。

```

sum=a[0];
for(int i=1;i<x;i++){
    if(a[i]==-2){    //加法
        sum=sum+a[i+1];
        i++;
    }
    else if(a[i]==-3){    //减法
        sum=sum-a[i+1];
        i++;
    }
    else if(a[i]==-1){    //“=”，停止读取
        QString str_ = QString::number(sum,'f',2);    //赋值输出，保留两位小数
        ui->result->setText(str_);
    }
}

```

还有一个问题，当我们计算完“ $5+5\times 6$ ”后，得到结果为25，此时我还要接着往下进行运算。这看似是一个连续计算的过程，但一旦进行“=”的操作，就是进入下一轮的计算了，之前的数据都会被清空。所以，数组 `arr` 中的数据清空准备进行下一轮运算，而“25”作为上一轮的运算结果，也要参与这一轮的运算，那么就把“25”赋值到数组 `arr` 中，当作这一轮的运算数据。

```

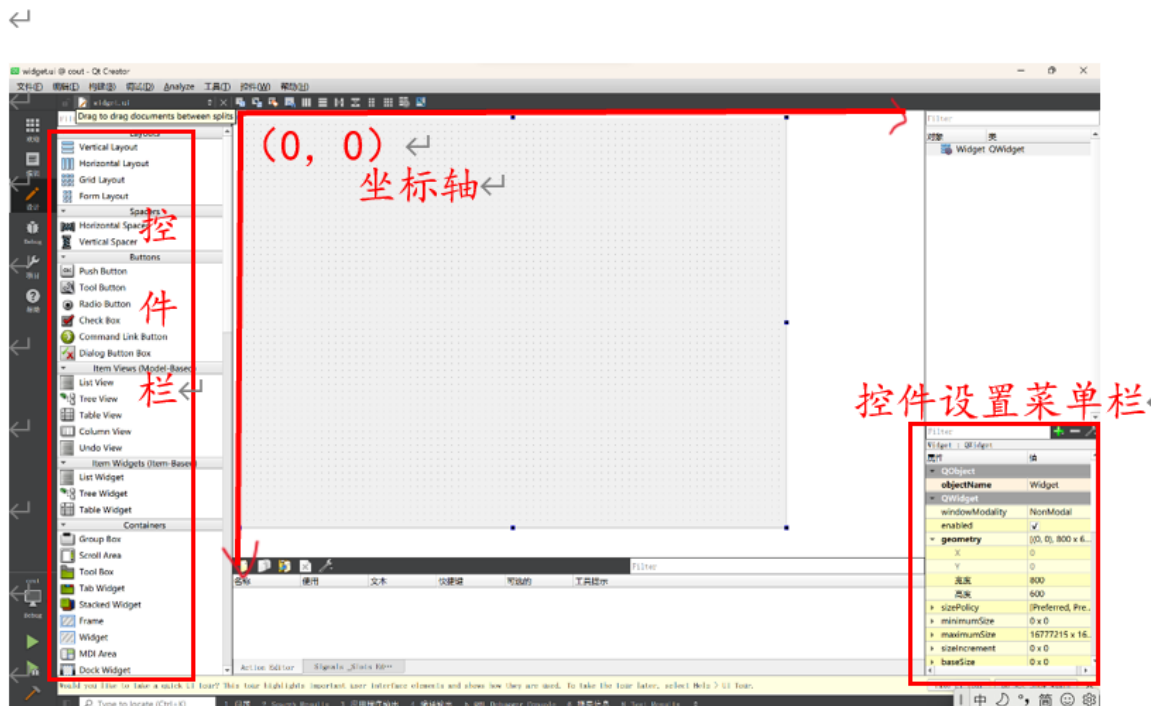
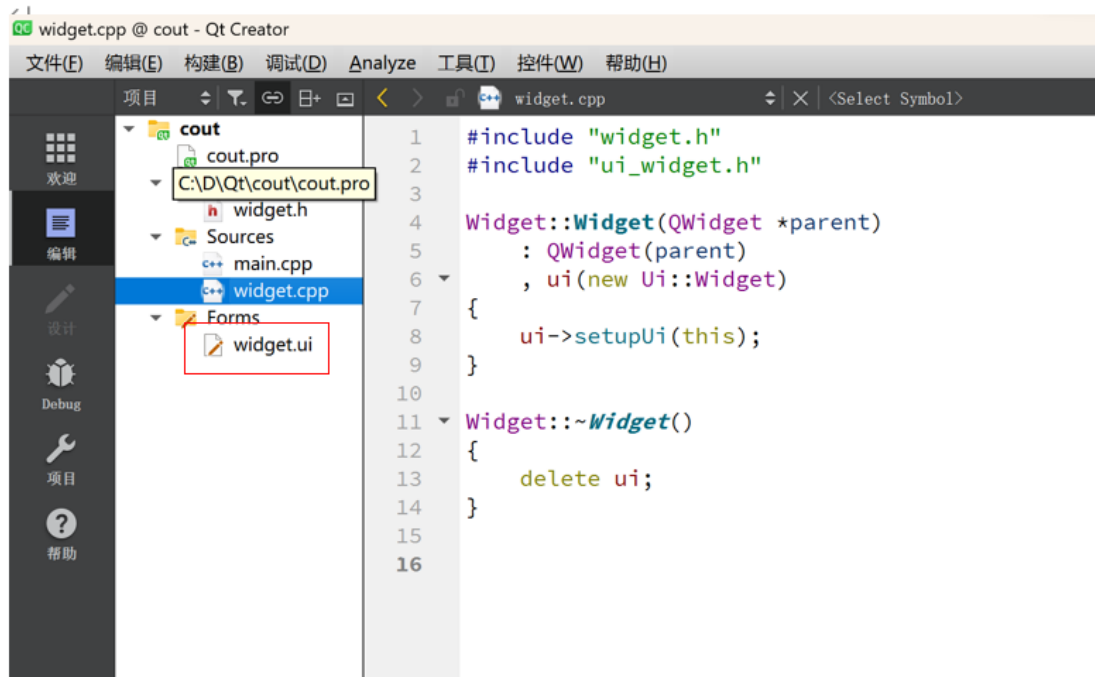
        else if(a[i]==-1){    //“=”，停止读取
            QString str_ = QString::number(sum,'f',2);    //赋值输出，保留两位小数
            ui->result->setText(str_);
            m=0;
            arr[m]=sum;
            m++;
            str=to_string(sum);
            for(int i=1;i<100;i++){
                arr[i]=-1;
            }
        }
    }
}

```

解决完主体程序运行的部分，将运算过程与结果输出到显示屏，即我们设定的文本框中就很容易了。

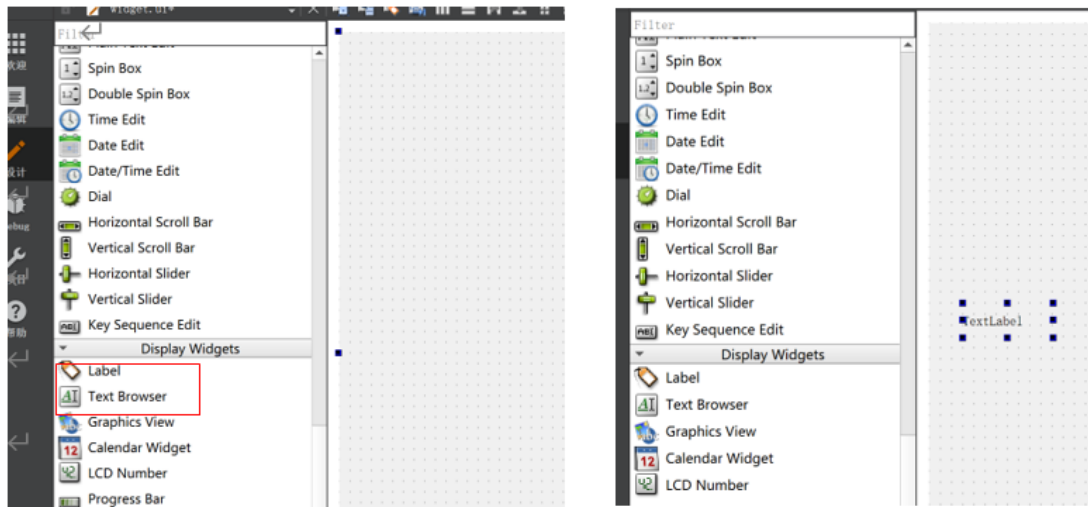
3.3 设计 QT 界面

打开 widget.ui



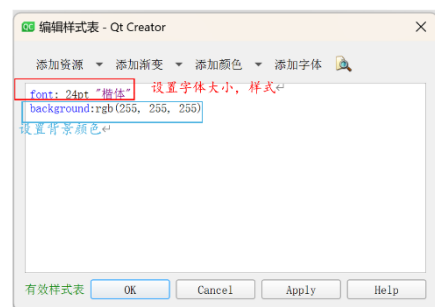
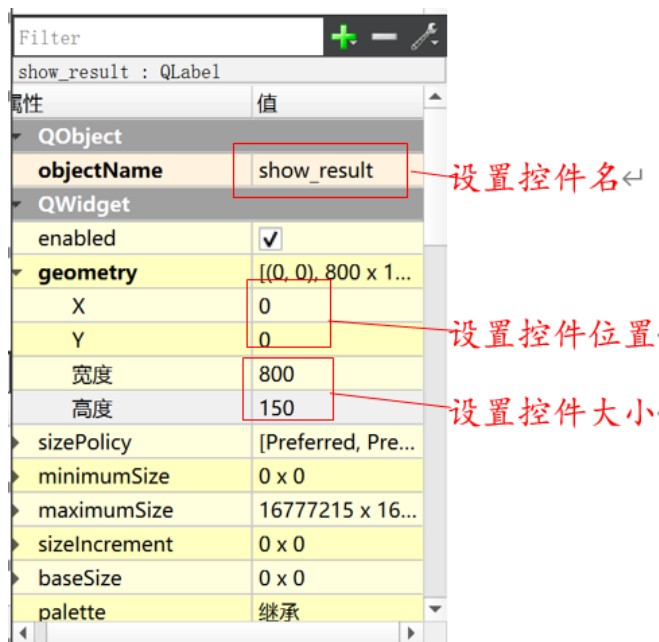
首先，我们先设置显示屏。

下拉控件栏，找到 Label 控件，点击并拖拽到右侧设计界面。



选中控件，并在右下角的控件菜单栏进行设置。

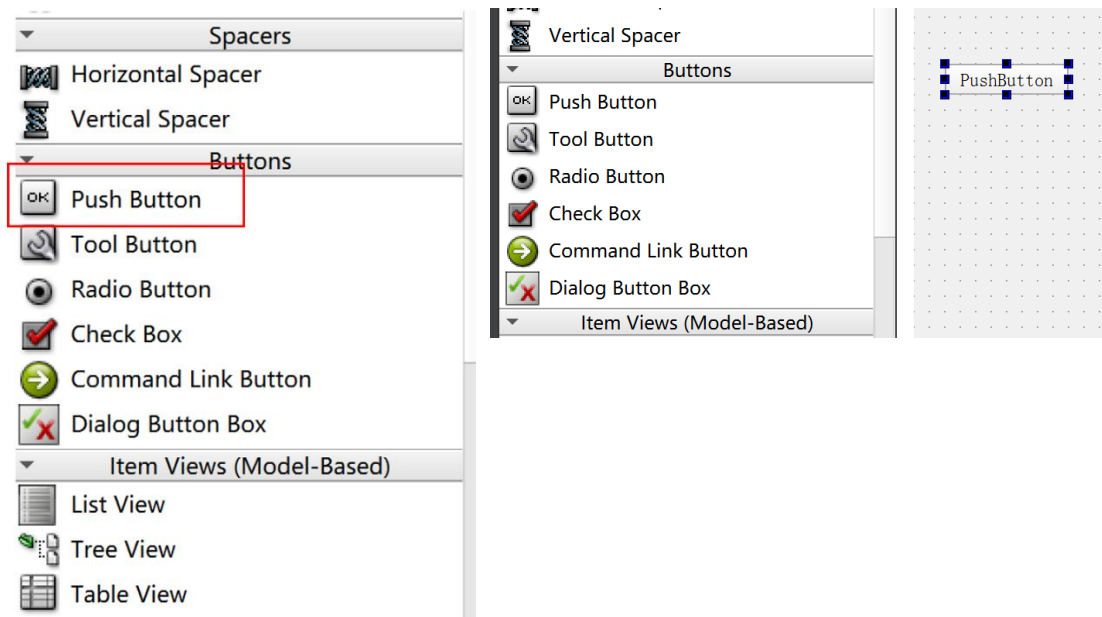
选中控件，右击鼠标，可以设定文本样式（字体样式，字体大小，文本排列，文本框背景……）



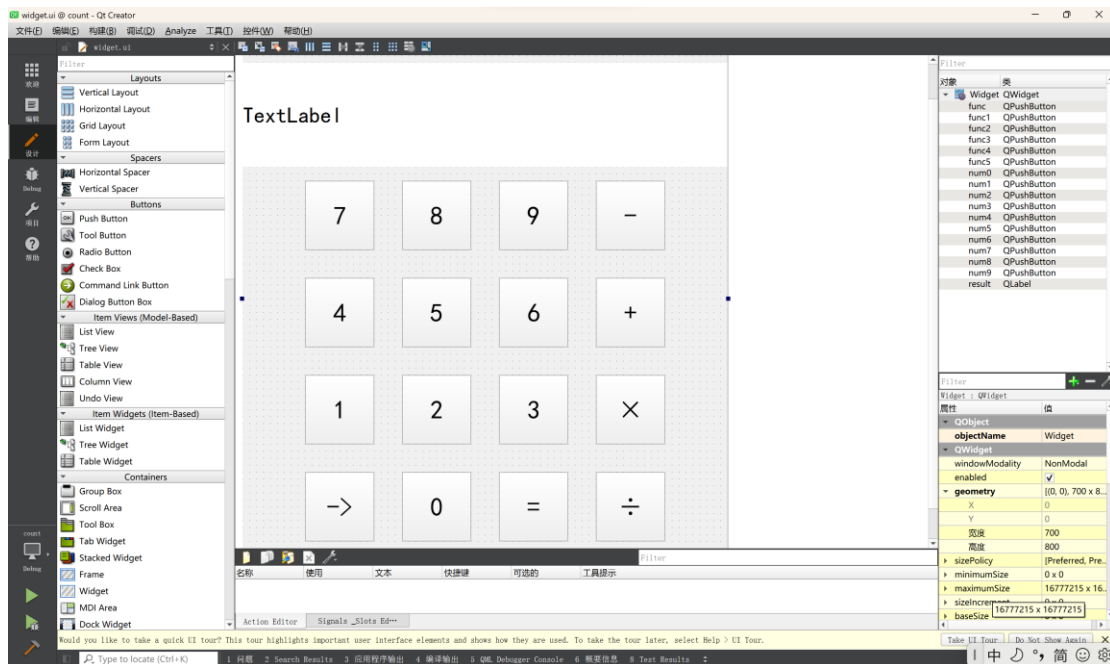
*注：关于控件位置的设置，也可以将控件拖拽至指定位置。

然后，我们来设置“0”~“9”，“+”“-”“×”“÷”“=”
“->”按钮。

下拉控件栏，找到 Push Button 控件，点击并拖拽到右侧设计界面。



调整好显示屏与按钮的大小和位置，界面设计初步完成。



3.4 信号与槽

信号和槽机制是 Qt 的核心特性，用于多个对象之间的通信，这也是 Qt 与其他框架最大的不同之处。Qt 的元对象系统是信号和槽实现的基础。

在 GUI 编程中，当更改一个小部件时，通常希望另一个小部件得到通知。希望任何类型的对象之间都能够相互通信。例如，如果用户单击关闭按钮，可能希望调用窗口的 `Close()` 函数。

其他软件工具包或框架可能使用回调机制实现这种通信机制。一个回调函数是一个指向一个函数的指针，所以如果想让一个处理函数通知一些事件，可以向处理函数传递一个指向另一个函数(回调函数)的指针，然后处理函数在适当的时候调用回调函数。虽然使用这种方法的成功框架确实存在，但回调可能不太直观，在确保回调参数类型的正确性上可能会存在问题。

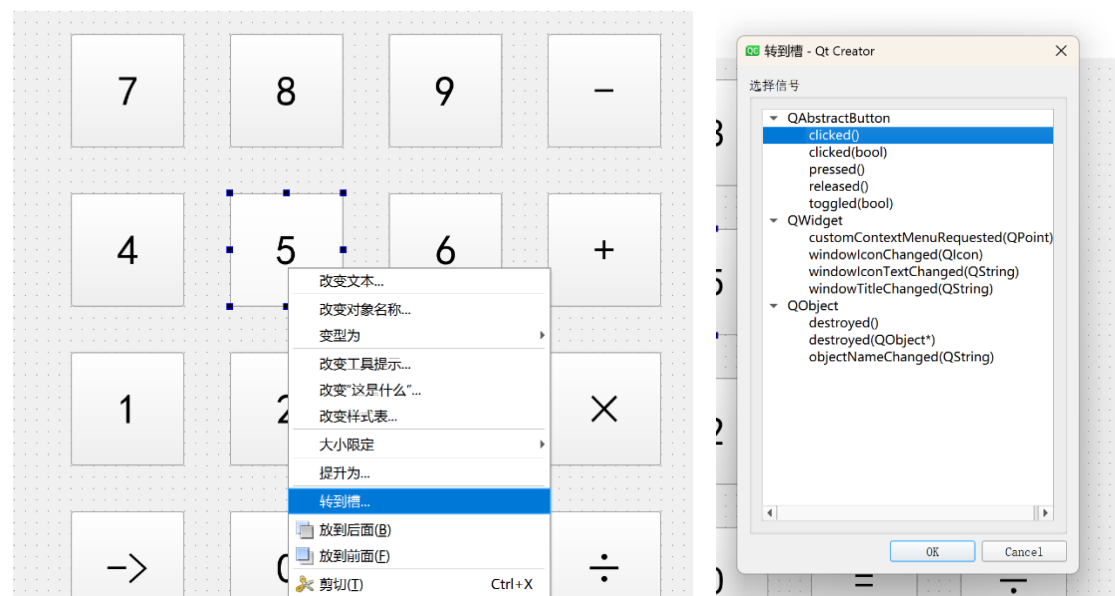
在 Qt 中，有一种回调技术的替代方法：那就是信号和槽机制。当特定事件发生时，会发出一个信号。Qt 的小部件中有许多预定义的信号，但我们可以将小部件子类化，向它们添加自定义的信号。槽是响应特定信号的函数。Qt 的小部件有许多预定义的槽函数，但是通常是子类化小部件并添加自己的槽函数，这样就可以处理与之相关联的信号了。【8】

那么，在整数四则计算这个例子中，当我们计算 $5 \times 6 = 30$ 时，先

点击按键“5”，然后显示屏上出现数字“5”。这个过程拆开来应该是：用户（信号发送者）->点击按键“5”（发送信号）->显示屏（接收信号）->显示输出“5”（反应结果，槽函数调用）。

一般来说，很多时候会使用，`QObject::connect()`来连接信号与槽函数，但由于在这里我们设计在 ui 界面已经设计好控件按钮，所以我们可以直接利用按钮与槽函数建立联系。

比如，我们希望点击按键“5”时显示屏出现数字“5”，那么先选中按键“5”，右击鼠标，点击“转到槽...”。



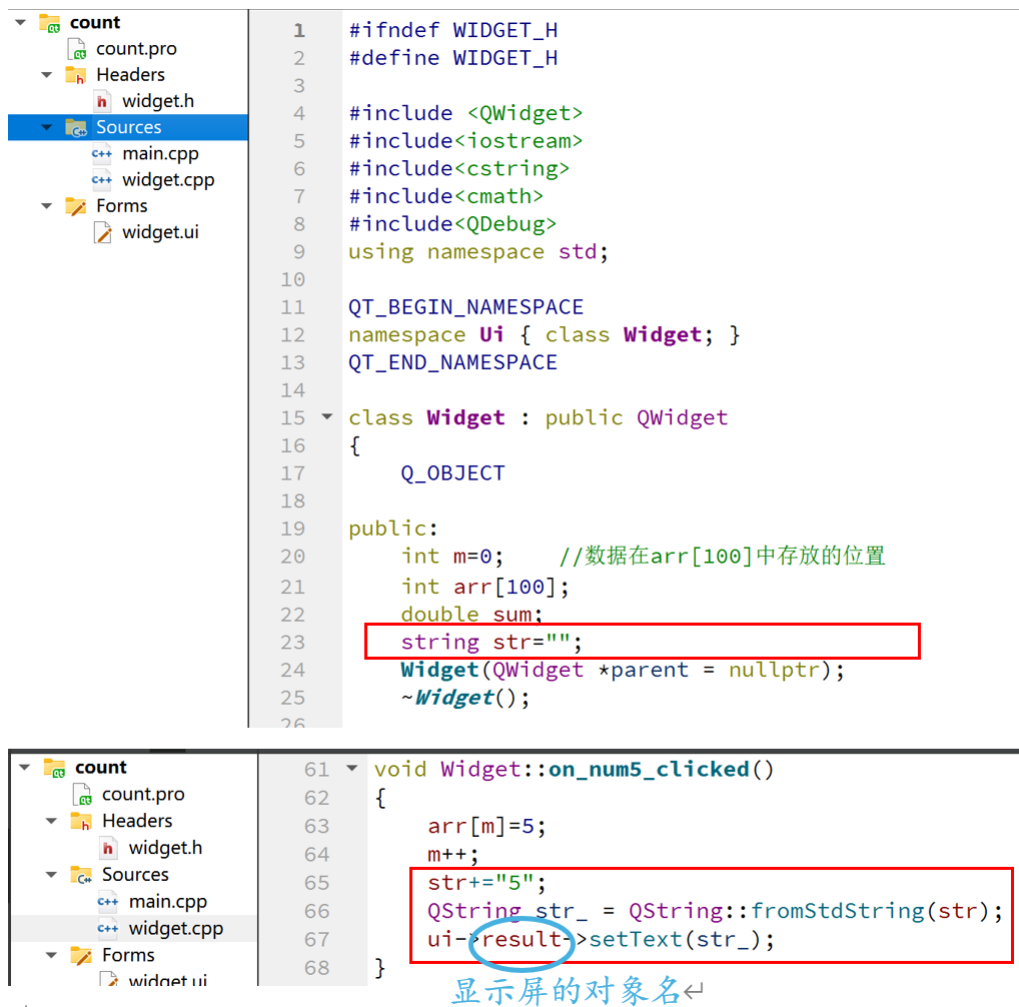
因为信号发出的种类很多，有的只要求单击，有的需要双击，在这里我们只是需要单击按键，选择 `clicked()` 就可以。

界面跳转到 `widget.cpp` 界面，生成按键“5”的槽函数，同时，在 `widget.h` 界面的“`private slots:`”下面，可以查看到该槽函数的声明^①。

^①注：槽函数必须写在“`private slots:`”下面才可以生效。



那么点击数字“5”，让显示屏显示 5，就要用到 `setText()` 函数，但是用 `Label` 进行输入输出，用到的数据类型都是 `QString`，所以所有数据类型都只有转成 `QString` 类型才可以进行输入输出。



顺便一提，若是要获取外部输入的数据，则需要用到 `text()` 函数，获取的信息的数据类型也是 `QString`，需要转化成所需要的数据类型。

3.5 案例总结

widget.h 中的代码如下：

```
#ifndef WIDGET_H
#define WIDGET_H

#include <QWidget>
#include<iostream>
#include<cstring>
#include<cmath>
#include<QDebug>
using namespace std;

QT_BEGIN_NAMESPACE
namespace Ui { class Widget; }
QT_END_NAMESPACE

class Widget : public QWidget
{
    Q_OBJECT

public:
    int m=0;    //数据在 arr[100]中存放的位置
    //int arr[100];
    double arr[100];
    double sum;
    string str="";
    Widget(QWidget *parent = nullptr);
    ~Widget();

private slots:
    void on_num0_clicked();
    void on_num1_clicked();
    void on_num2_clicked();
    void on_num3_clicked();
    void on_num4_clicked();
    void on_num5_clicked();
    void on_num6_clicked();
    void on_num7_clicked();
```

```

void on_num8_clicked();
void on_num9_clicked();

void on_func1_clicked();
void on_func2_clicked();
void on_func3_clicked();
void on_func4_clicked();

void on_func_clicked();

void on_func5_clicked();

private:
    Ui::Widget *ui;
};
#endif // WIDGET_H

```

widget.cpp 中的代码如下:

```

#include "widget.h"
#include "ui_widget.h"

Widget::Widget(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::Widget)
{
    ui->setupUi(this);
    ui->result->setText("");
    for(int i=0;i<100;i++){
        arr[i]=-1;
    }
}

Widget::~Widget()
{
    delete ui;
}

```

```

void Widget::on_num0_clicked()
{
    arr[m]=0;
    m++;
    str+="0";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}
void Widget::on_num1_clicked()
{
    arr[m]=1;
    m++;
    str+="1";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}
void Widget::on_num2_clicked()
{
    arr[m]=2;
    m++;
    str+="2";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}
void Widget::on_num3_clicked()
{
    arr[m]=3;
    m++;
    str+="3";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}
void Widget::on_num4_clicked()
{
    arr[m]=4;
    m++;
    str+="4";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}
void Widget::on_num5_clicked()
{
    arr[m]=5;
    m++;

```

```

        str+="5";
        QString str_ = QString::fromStdString(str);
        ui->result->setText(str_);
    }
void Widget::on_num6_clicked()
{
    arr[m]=6;
    m++;
    str+="6";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}
void Widget::on_num7_clicked()
{
    arr[m]=7;
    m++;
    str+="7";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}
void Widget::on_num8_clicked()
{
    arr[m]=8;
    m++;
    str+="8";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}
void Widget::on_num9_clicked()
{
    arr[m]=9;
    m++;
    str+="9";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}

void Widget::on_func1_clicked()
{
    arr[m]=-2;
    m++;
    str+=" ";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}

```

```

}
void Widget::on_func2_clicked()
{
    arr[m]=-3;
    m++;
    str+="-";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}void Widget::on_func3_clicked()
{
    arr[m]=-4;
    m++;
    str+="×";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}void Widget::on_func4_clicked()
{
    arr[m]=-5;
    m++;
    str+="÷";
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
}

void Widget::on_func_clicked()
{
    arr[m]=-1;
    m++;
    int b=0;
    double a[100];
    //int a[100];  //存放处理过后的数据的数组
    int x=0;
    for(int i=0;i<m;i++){

        if(arr[i]>=-5&&arr[i]<0){    //遇到运算符
            int y=0;
            int num=0;    //处理后的数据

            for(int j=i-1;j>=i-b;j--){    //将零散的数字组成一个新的
多位数
                num=num+arr[j]*pow(10, y);
                y++;
            }
            a[x]=num;    //将处理过的数据放入数组 a 中

```



```

        x++;
        a[x]=arr[i];    //arr[i]存放的数据代表某一运算符，直接赋
值到数组 a 中，无需处理
        x++;
        b=0;
    }
    else if(arr[i]>=0){    //计算上一运算符到下一运算符之间数字个
数
        b++;
    }
}
for(int i=1;i<x-1;i++){
    if(a[i]==-4){    //乘法运算
        a[i]=a[i-1]*a[i+1];    // “×” 号前后数字相乘计算
        a[i-1]=-10;
        a[i+1]=-10;
    }
    else if(a[i]==-5){    //除法运算
        a[i]=a[i-1]/a[i+1];    // “÷” 号前后数字相乘计算
        a[i-1]=-10;
        a[i+1]=-10;
    }
}
for(int i=0;i<x-1;i++){
    for(int j=0;j<x-i-1;j++){
        if(a[j]==-10){
            swap(a[j],a[j+1]);
        }
    }
}
sum=a[0];
for(int i=1;i<x;i++){
    if(a[i]==-2){    //加法
        sum=sum+a[i+1];
        i++;
    }
    else if(a[i]==-3){    //减法
        sum=sum-a[i+1];
        i++;
    }
    else if(a[i]==-1){    // “=” ， 停止读取
        QString str_ = QString::number(sum,'f',2);    //赋值输出，保留两位小数
        ui->result->setText(str_);

```

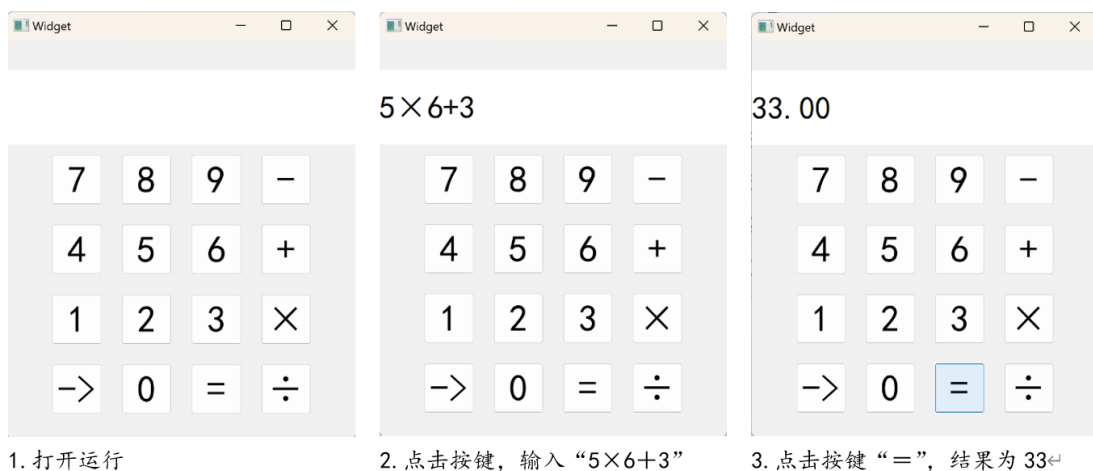
```

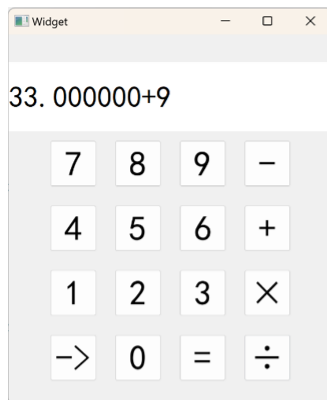
        m=0;
        arr[m]=sum;
        m++;
        str=to_string(sum);
        for(int i=1;i<100;i++){
            arr[i]=-1;
        }
    }
}

void Widget::on_func5_clicked()
{
    if(m==1){
        str="";
    }
    else{
        str=str.substr(0, str.length()-1);
    }
    QString str_ = QString::fromStdString(str);
    ui->result->setText(str_);
    m--;
    arr[m]=-1;
}

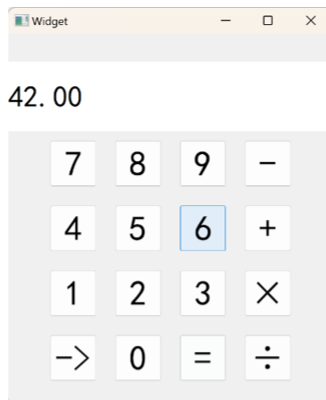
```

运行结果如下：

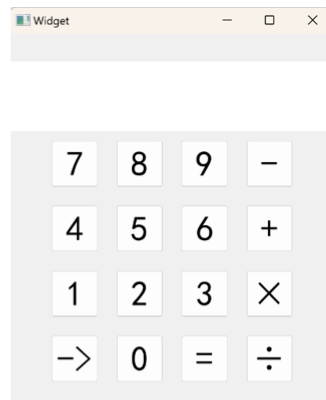




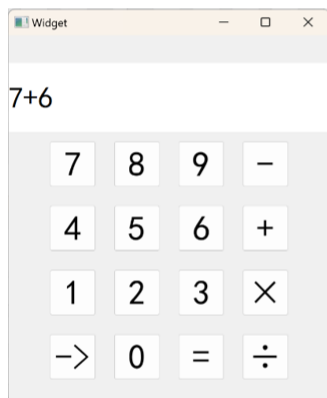
4. 点击按键，输入“+9”



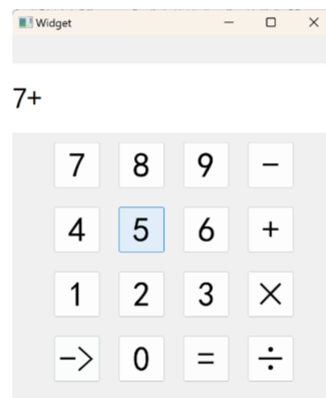
5. 点击按键“=”，结果为42



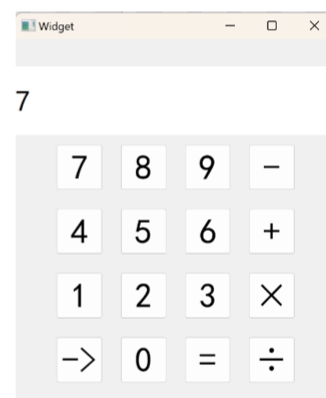
6. 点击按键“<->”，结果清零



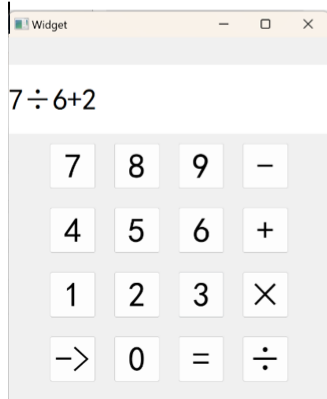
7. 点击按键，输入“7+6”



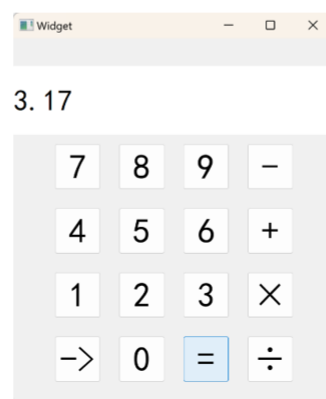
8. 点击按键“<->”，结果变为“7+”



9. 点击按键“<->”，结果变为“7”



10. 点击按键，输入“7÷6+2”



11. 点击按键“=”，结果为3.17

4. QDebug 的使用

在项目的开发中，当我们调试程序时，最常用的就是打印信息了，Qt 为我们提供了非常方便的打印信息类 QDebug。

使用 qDebug 的最常用法，就像使用 `std::cout` 一样，直接在重定向操作符 “<<” 后面加上字符串，这种输出方式是需要包含头文件 `#include<QDebug>` 的。比如：`qDebug() << "Hello" << 123`。

`qDebug()` 还可以打印 Qt 中的基本类型，例如 `QString`，`QByteArray`，`QDate`，`QRectF`，`QVariantHash`，`QFont` 等等。【9】

那么在上面的案例中，如果我要查看每次点击按键是否有往数组 `arr` 中输入相应功能的编号，则可以用 `qDebug()` 来验证。比如：插入一下的代码，在当我们输入 “5×6” 时，点击按键 “=” 时，控制台会进行输出，以此检验代码的正确性。

```
void Widget::on_func_clicked()    //点击“=”
{
```

```
    arr[m] = -1;
    m++;
    for(int i=0; i<m; i++){
        qDebug() << arr[i];
        qDebug() << "-----";    //分界
    }
    int b=0;
    double a[100];
```

```
22:37:09: Starting C:\Qt\build-calculate-Desktop_Qt_5_14_2_MinGW_32_bit-Debug\debug\count.exe ...
```

```
5  数字 “5” ←
```

```
-4 运算符 “×” ←
```

```
6  数字 “6” ←
```

```
-1 运算符 “=” ←
```

后记

以上便是我近两周学习 QT 的主要成果，由于篇幅限制，没有完整讲述其他的内容，例如：QT 运行时中文乱码应如何解决，在 QT 中开启新窗口如何保存原窗口指针以便返回原窗口的问题……

这次技术学习培养了我自主查找资料教程的能力，也拓宽了我的知识面，对于 C++ 的可视化有了进一步的了解，受益匪浅。这让我深刻地知道，学习更应该自己去找资源，主动了解。学习一门未知的技能看着困难重重，实际上确实如此，但其困难程度却远没有想象中的大。

一开始，看到课程报告要求的那一刻，我几乎坚信自己难以完成任务，无论是阅读一篇全英顶刊论文还是学习一项技术，我都是一头雾水。但这一个月来，我也确实做到了。机缘巧合之下，我真阅读了一篇关于联邦梯度学习以恢复训练数据的全英论文（《Deep Leakage from Gradients》），由于基础问题，我并没有很理解其中内容，但是这对我来说是一种经验的积累。而从一开始为了期末课设作业学习的 QT 技术，也让我产生了浓厚的兴趣，希望可以更加深入地学习。

之前专业有过竞赛指导的沙龙，我有幸在会议的最后向老师提问，我说我作为一个计科专业的小白，总是会迷茫地不知道往哪个方向努力去做的更好。李培培老师现身说法地告诉我们，即使基础不够，对计算机专业了解还没有很深刻，也要尽自己的努力做的更

好，做一个不够就再做一个。所以，我从最初的不自信与迷茫到现在自主学习有了一点点的小收获，用心请教，认真学习总会有进步的。

万事开头难，但是只要肯努力，肯研究，总会有你意想不到的收获，从不可能到可能从来不是隔着不可逾越的天堑。

*注：以上内容的所有代码均为个人所写，不足之处还望指正包含。

【1】【5】【6】引用自 CNSD—百无一用狂书生《Qt 介绍》--2018-04-26

【2】【4】引用自 CNSD—百无一用狂书生《Qt 简介（二）》--2018-04-26

【3】引用自知乎—千锋教育《图形用户界面（GUI）入门详解》--2019-05-28

【7】引用自 CNSD—Love 朴啾啾—《Qt5.14.2 安装、配置及测试（Win10）》--2022-05-15

【8】引用自知乎—iriczha0《Qt 一篇全面的信号和槽函数机制总结》--2023-02-05

【9】引用自 CNSD—四九城小白~阿勋《Qt 中关于 qDebug 的使用》--2019-05-05